

Kubernetes Operators in Java

Quarkus + JOSDK on OpenShift



QUARKUS



Who this is for

- Quarkus developers — REST, CDI, extensions, `quarkus:dev`
- Platform engineers curious about OLM and OperatorHub
- Integration folks who want Camel on OpenShift without reinventing the wheel

If you can write a `@ApplicationScoped` bean, you can write a Reconciler.

Agenda

Block	Content
<u>Theory</u>	Operators, JOSDK / QOSDK, CRD, reconcile, OLM bundle
<u>Live demo · part 1</u>	Joke 101 — Quarkus scaffold + <u>operator-sdk</u> on OpenShift Local
<u>Live demo · part 2</u>	OpenShift Integration Operator — <u>ephemeral</u> Quick Try

*Same OLM path → anyone can publish to the community catalog
(OperatorHub)*

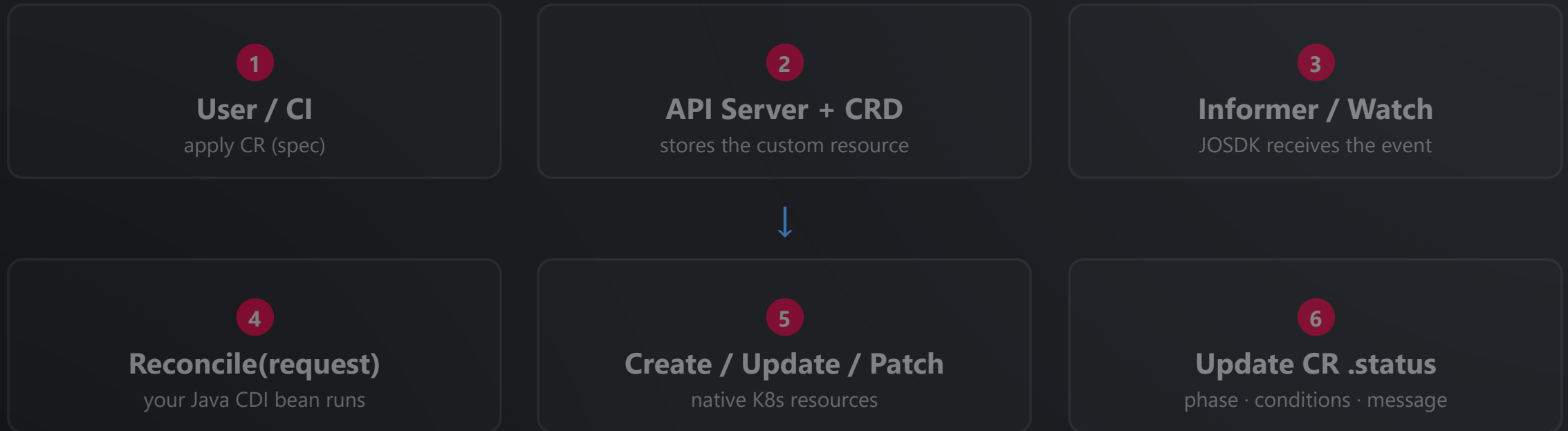
Why Kubernetes Operators?

Problem: Stateful apps on K8s need human ops — install, upgrade, backup, failovers.

Solution: Encode ops knowledge in software that runs **inside** the cluster.

- Extend the API with **Custom Resources (CRs)**
- A **controller** watches CRs and drives real resources (Deployments, Services, ...)
- **Declarative:** users set `spec`; operator converges cluster to desired state

Operator anatomy



Key idea: idempotent reconcile — safe to run again and again · loop repeats

The open-source stack



- **Quarkus** — fast startup, CDI, native image option
- **QOSDK** — Quarkus extension wrapping Java Operator SDK
- **Fabric8** — Kubernetes client (generated from CRDs)
- **OLM** — install / upgrade operators via catalog

You don't need Go

	Go (controller-runtime)	Java (QOSDK)
Ecosystem default	Yes	Growing
Team skills	Go required	<u>Java / Quarkus</u>
Dev experience	controller-gen	<u>code.quarkus.io</u> , <code>quarkus:dev</code>
OLM bundles	operator-sdk	<u>bundle-generator extension</u>

Quarkus Club: same toolchain you already use for microservices

Scaffold: code.quarkus.io + Quarkus CLI

1. Go to code.quarkus.io → search [qosdk](#) or [operator](#)
2. Select [Quarkus Operator SDK](#) extension
3. Optionally add [OLM Bundle Generator](#)
4. Generate & download — or:

```
quarkus create app org.acme:my-operator \
  --extension='quarkus-operator-sdk,quarkus-operator-sdk-bundle-generator'
cd my-operator
./mvnw quarkus:dev
```


Define your Custom Resource

Java types in `src/main/java/ ... /v1/` — JOSDK generates CRD YAML on build:

```
@Group("platform.io")
@Version("v1alpha1")
@Kind("IntegrationFlow")
public class IntegrationFlow extends CustomResource<
    IntegrationFlowSpec, IntegrationFlowStatus> {}
```

- **Spec** — desired state (user-facing)
- **Status** — observed state (operator-written)

Reference: [Joke sample](#)

The Reconciler — a CDI bean

```
@ControllerConfiguration(  
    csvMetadata = @CSVMetadata(  
        displayName = "Integration Operator",  
        bundleName = "openshift-integration-operator"))  
public class IntegrationFlowReconciler  
    implements Reconciler<IntegrationFlow> {  
  
    @Override  
    public UpdateControl<IntegrationFlow> reconcile(  
        IntegrationFlow resource, Context<IntegrationFlow> context) {  
        // compare spec → create/update Deployments, Services...  
        // update status phase: Pending → Running / Error  
        return UpdateControl.patchStatus(resource);  
    }  
}
```

Reconciliation loop · best practices

1. Level-triggered, not edge-only — reconcile on timer + watch
2. Idempotent — same spec → same outcome every time
3. Owner references — child resources garbage-collected with CR
4. Status conditions — Ready, Degraded, Progressing
5. Don't put secrets in spec — reference Secret names

Ephemeral mode in our demo uses ownerReferences so deleting the CR cleans up worker Deployments

OLM bundle generation

Add extension: `quarkus-operator-sdk-bundle-generator`

```
mvn clean package \
  -Dquarkus.container-image.build=true \
  -Dquarkus.container-image.registry=quay.io \
  -Dquarkus.container-image.group=maximilianopizarro \
  -Dquarkus.operator-sdk.bundle.channels=alpha
```

Output: `target/bundle/<name>/manifests/` + `bundle.Dockerfile`

Docs: [Deploy with OLM](#)

Deploy to OpenShift · test path

```
# Validate bundle
operator-sdk bundle validate target/bundle/openshift-integration-operator \
  --select-optional name=operatorhub

# Install from published bundle (community)
operator-sdk run bundle \
  quay.io/maximilianopizarro/openshift-integration-operator-bundle:v0.8.2 \
  --namespace openshift-integration \
  --install-mode AllNamespaces \
  --timeout 10m
```

Or: **OperatorHub** in OpenShift Console → Install

Same bundle path → PR to [community-operators](#) → catalog

From skeleton to product

After Joke 101 — a real operator already on OperatorHub

OpenShift Integration Operator — QOSDK · Apache 2.0 · community catalog

- GitHub: [maximilianoPizarro/openshift-integration-operator](https://github.com/maximilianoPizarro/openshift-integration-operator)
- **IntegrationFlow** CR — Camel routes, Kaoto design, ephemeral or GitOps
- Console dynamic plugin · 250+ flow templates

Today (part 2): **ephemeral Quick Try only** — GitOps is in the repo, not live today

Demo environment · try it at home

Path: [Podman Desktop](#) + [OpenShift Local](#) · free [Red Hat Developer](#) account

1. [Joke 101](#) — internal registry + `operator-sdk run bundle` → [tutorial Getting Started](#)
2. [Next](#) — Integration Operator from OperatorHub (ephemeral)

Prefer pure OSS? [kind](#) works for Joke / operator runtime — no OpenShift Console plugin UX.

Live demo · part 1

Joke 101 on OpenShift Local

SCAFFOLD · INTERNAL REGISTRY · OPERATOR-SDK RUN BUNDLE ·
JOKEREQUEST

Demo · Joke 101 · operator-sdk

```
# From joke-operator/ - images → OpenShift Local registry
mvn clean package -Dquarkus.container-image.build=true \
  -Dquarkus.container-image.registry=default-route-openshift-image-registry.apps-crc.testing \
  -Dquarkus.container-image.group=openshift-operators

operator-sdk run bundle \
  default-route-openshift-image-registry.apps-crc.testing/openshift-operators/joke-operator-bundle:1.0.0-SNAPSHOT \
  --namespace openshift-operators --timeout 10m

kubectl apply -f k8s/jokerequest-sample.yaml
```

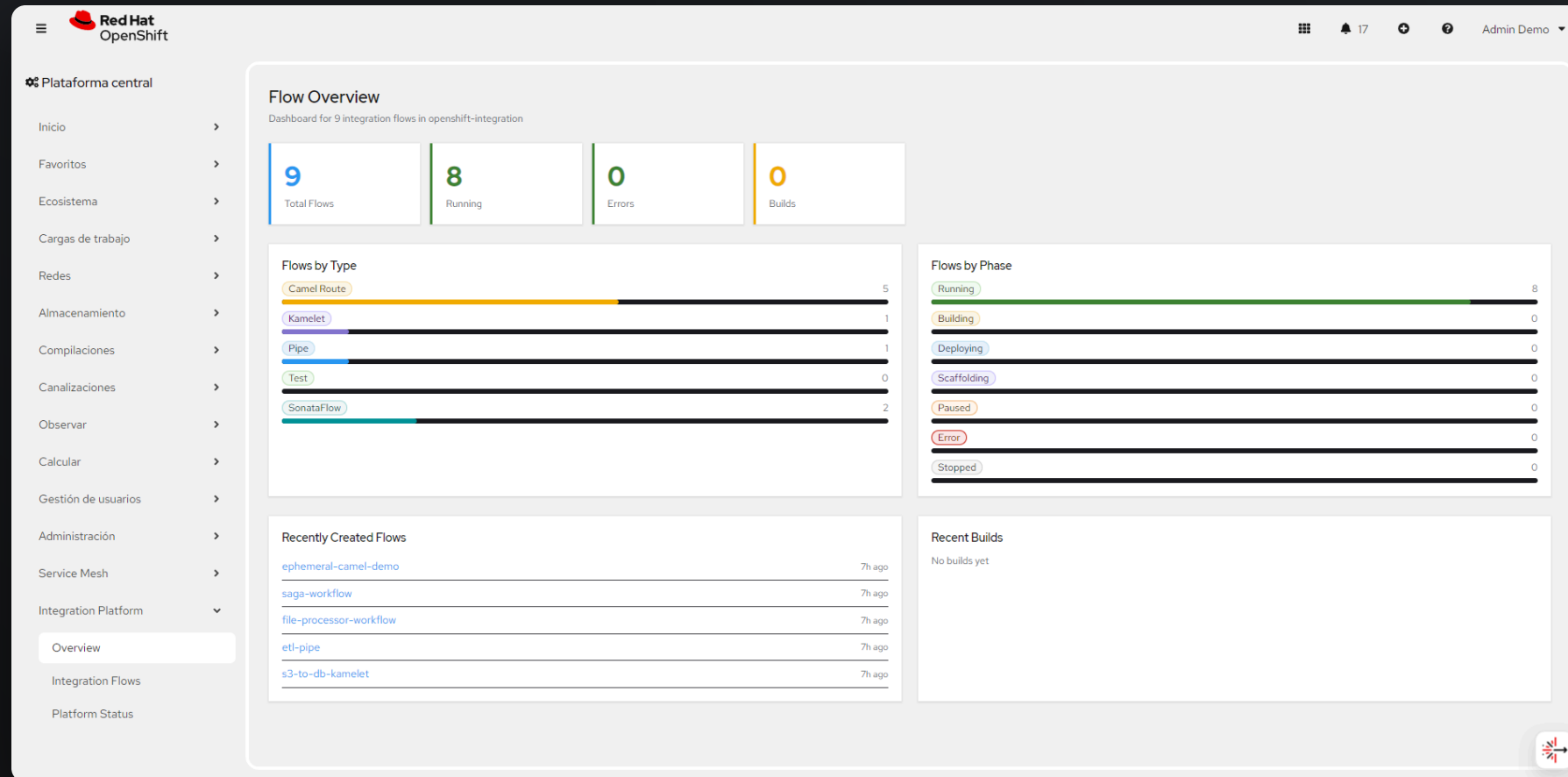
Full steps: [tutorial repo](#) · [docs](#)

Live demo · part 2

OpenShift Integration Operator

COMMUNITY CATALOG · CONSOLE PLUGIN · KAOTO · EPHEMERAL
QUICK TRY

Demo · Install from OperatorHub



Operators → OperatorHub → "OpenShift Integration Operator" → Install

Demo · Console plugin

The screenshot displays the Red Hat OpenShift console interface. On the left, the 'Plataforma central' sidebar is visible, with 'Integration Platform' selected under the 'Observar' section. The main content area shows the 'Integration Flows' page, which lists 9 flows in the 'openshift-integration' namespace. The flows are categorized by type (Camel Route, Kamelet, Pipe, SonataFlow) and phase (Expired, Running). The table below provides a detailed view of these flows.

Name	Type	Mode	Phase	Clusters	Condi...	Repository	Resources	Age	Actions
ephemeral-camel-demo	Camel Route	Ephemeral · expired	Expired	—	●●●	—	Logs Pods Pipelines	7h	Actions
error-handling-dlq	Camel Route	GitOps	Running	1/1	●●●	user/error-handling-dlq / @e48cb45	Logs Pods Builds Pipelines Argo	7h	Actions
etl-pipe	Pipe	GitOps	Running	1/1	●●●	user/etl-pipe / @f8936da	Logs Pods Builds Pipelines Argo	7h	Actions
file-processor-workflow	SonataFlow	GitOps	Running	1/1	●●●●	user/file-processor-workflow / @ad3fd37	Logs Pods Builds Pipelines Argo	7h	Actions
order-routing-cbr	Camel Route	GitOps	Running	1/1	●●●	user/order-routing-cbr / @f39ad77	Logs Pods Builds Pipelines Argo	7h	Actions
parallel-enrichment	Camel Route	GitOps	Running	1/1	●●●	user/parallel-enrichment / @585ae46	Logs Pods Builds Pipelines Argo	7h	Actions
rest-to-kafka	Camel Route	GitOps	Running	1/1	●●●	user/rest-to-kafka / @9548d1e	Logs Pods Builds Pipelines Argo	7h	Actions
s3-to-db-kamelet	Kamelet	GitOps	Running	1/1	●●●	user/s3-to-db-kamelet / @5c7b4b8	Logs Pods Builds Pipelines Argo	7h	Actions
saga-workflow	SonataFlow	GitOps	Running	1/1	●●●●	user/saga-workflow / @049a311	Logs Pods Builds Pipelines Argo	7h	Actions

Integration Platform in the OpenShift Console navigation

Demo · Create ephemeral IntegrationFlow

```
apiVersion: platform.io/v1alpha1
kind: IntegrationFlow
metadata:
  name: ephemeral-camel-demo
  namespace: openshift-integration
spec:
  deploymentMode: EPHEMERAL
  integrationType: CAMEL_ROUTE
  ephemeral:
    ttlSeconds: 3600
  kaotoDesign: |
    - route:
        from:
          uri: "timer:hello?period=5000"
          steps:
            - log: { message: "Hello from Quarkus Club!" }
```

Apply: `oc apply -f k8s/examples/09-ephemeral-demo.yaml`

Demo · Kaoto visual designer

The screenshot displays the Red Hat OpenShift console with the Kaoto visual designer embedded. The console's left sidebar shows the 'Plataforma central' menu with various system management options. The main area is divided into three sections: a top header for the 'parallel-enrichment' flow, a central visual editor, and a right-hand telemetry and resources panel.

Flow Header: Shows 'parallel-enrichment' in a 'Running' state with a 'GitOps' button. It includes 'Pause', 'Stop', and 'Save Design' controls. The engine is 'CAMEL_ROUTE' on the 'main' branch.

Visual Editor: The central workspace shows a Camel route diagram. It starts with a 'platform-http' connector, followed by a 'log' step, and then a 'multicast' step that branches into three parallel 'direct' routes. The interface includes a toolbar with 'YAML', 'Camel Route', and 'parallel-enrichment 1/1' tabs, along with a '+ New' button and a 'Camel Main 4.14.2.redhat-00019' version selector.

Right Panel: Contains 'Node Telemetry' showing metrics for 'http-source-1', 'log-processor', 'kafka-sink', and 'transform-1'. Below this is a 'Resources' section with links for Logs, Runtime Pods, Build Pods, PipelineRuns, ArgoCD App, and Git Repository. The 'Target Clusters' section shows 'None'. The 'State' is 'running'. The 'Last Commit' is '585ae46'. The 'Message' states: 'ApplicationSet reconciled, tracking 1 cluster deployment(s)'.

Drag-and-drop Camel routes · embedded in the console plugin

Ephemeral architecture (today's path)

OpenShift Console (Plugin + Kaoto)

|

▼

IntegrationFlow CR (deploymentMode: EPHEMERAL)

|

▼

Operator Reconciler → EphemeralWorkerImageResolver

|

▼

Worker Deployment

|

▼

precompiled Camel Quarkus image (Quay)

|

▼

Logs / OTel / SSE → Console (Flow Logs, status)

GitOps (Gitea · Tekton · Argo CD) — in repo, not live today

Demo · Visual diagram & status

The screenshot displays the Red Hat OpenShift console interface. On the left, a sidebar menu lists various system components. The main area shows the 'saga-workflow' in a 'Running' state, with a 'GitOps' button and 'Pause', 'Stop', and 'Save Design' controls. Below this, a visual state machine diagram illustrates the workflow's logic, starting with 'InitState' and transitioning through 'CheckResult', 'SuccessState', 'ErrorState', 'processAction', 'notifySuccess', and 'handleError'. To the right of the diagram, a YAML editor shows the configuration for the workflow, including its ID, version, and state definitions. Further right, a 'Node Telemetry' table provides performance metrics for different components, and a 'Resources' section offers links to logs and other tools. The 'State' section indicates the workflow is 'running', and the 'Last Commit' is shown as '@49a311'.

Red Hat OpenShift

Plataforma central

- Inicio
- Favoritos
- Ecosistema
- Cargas de trabajo
- Redes
- Almacenamiento
- Compilaciones
- Canalizaciones
- Observar
- Calcular
- Gestión de usuarios
- Administración
- Service Mesh
- Integration Platform

← Back to Flows

saga-workflow Running GitOps

Engine: SONATAFLOW Branch: main

Pause Stop Save Design

YAML · LINES 7-8

```
1 id: saga-workflow
2 version: "1.0"
3 specVersion: "0.8"
4 name: Saga Workflow
5 start: InitState
6 states:
7   - name: InitState
8     type: operation
9   actions:
10     - functionRef:
11       refName: processAction
12     transition: CheckResult
13   - name: CheckResult
14     type: switch
15   dataConditions:
16     - condition: "${ .result == true }"
17     transition: SuccessState
18   defaultCondition:
19     transition: ErrorState
20   - name: SuccessState
21     type: operation
22   actions:
23     - functionRef:
24       refName: notifySuccess
25   end: true
26   - name: ErrorState
27     type: operation
28   actions:
29     - functionRef:
30       refName: handleError
31   end: true
32 functions:
33   - name: processAction
34     type: expression
35     operation: ".result = true"
```

Node Telemetry

http-source-1	85ms	1774rpm	●
log-processor	15ms	2584rpm	●
kafka-sink	34ms	305rpm	●
transform-1	177ms	3499rpm	●

Updated: 22:41:50

Resources

- Logs
- Runtime Pods
- Build Pods
- PipelineRuns
- ArgoCD App
- Git Repository
- SonataFlow Console
- SonataFlow CR

Target Clusters

None

State

running

Last Commit

@49a311

Message

ApplicationSet reconciled, tracking 1 cluster deployment(s)

Reconciler wrote status · diagram reflects route structure

Demo · Watch reconciliation

```
oc get integrationflow -n openshift-integration -w
```

```
oc get pods -n openshift-integration -l integrationflow=ephemeral-camel-demo
```

```
oc logs -f deploy/iflow-ephemeral-camel-demo-worker -n openshift-integration
```

Expect: **phase Running** · pod **2/2 or 1/1 Ready** · logs every 5s

Demo · Flow logs in console

The screenshot displays the Red Hat OpenShift console interface. On the left is a sidebar menu with options like 'Inicio', 'Favoritos', 'Ecosistema', 'Cargas de trabajo', 'Redes', 'Almacenamiento', 'Compilaciones', 'Canalizaciones', 'Observar', 'Calcular', 'Gestión de usuarios', 'Administración', 'Service Mesh', and 'Integration Platform'. The main area shows the 'demo-test' flow, which is currently 'Paused' and has an 'Ephemeral- 58m left' status. Below the flow name are tabs for 'Visual Flow', 'Kaoto Designer', 'YAML Editor', 'History', 'Dependencies', 'Spec', 'Logs' (selected), and 'Status'. The 'Logs' tab shows a list of log entries with timestamps and messages, such as 'Apache Camel Quarkus 3.36.0 is starting' and 'Hello Integration Platform'. On the right side of the console, there are several panels: 'Node Telemetry' showing metrics for 'http-source-1', 'log-processor', 'kafka-sink', and 'transform-1'; 'Resources' with links to 'Logs' and 'Runtime Pods'; 'Target Clusters' showing 'local'; 'State' showing 'paused'; 'Ephemeral Worker' showing 'deployment/lflow-demo-test-worker'; 'Expires At' showing '5/6/2026, 23:43:30'; and 'Message' showing 'Ephemeral flow paused by user'.

Live pod log streaming · container selector · follow mode

Demo · Platform status

The screenshot shows the Red Hat OpenShift Platform Status dashboard. The top navigation bar includes the Red Hat OpenShift logo, a hamburger menu, and user information (Admin Demo). The left sidebar lists navigation options under 'Plataforma central', with 'Platform Status' selected. The main content area is titled 'Platform Status' and shows '6/7 services healthy - Last checked: 22:39:04'. Below this, five summary cards display: 9 Integration Flows, 0 PipelineRuns, 209 ArgoCD ApplicationSets, 1 Ephemeral (Quick Try) with a 'Create Quick Try flow' link, and 6/7 Services Healthy. A 'Refresh' button is in the top right. The 'External Services' section lists several services with their health status and replica counts: Integration Operator (HEALTHY, 1/1 replicas), Console Plugin (HEALTHY, 1/1 replicas), Kaoto Designer (HEALTHY, 1/1 replicas), Gitea (Git Server) (HEALTHY, 1 replicas in gitea namespace), ArgoCD (GitOps) (HEALTHY, Server: 1 replicas), Tekton Pipeline (ERROR, Pipeline not found), and OpenTelemetry Collector (HEALTHY, 1 replicas). A 'Platform Pods (19)' section is partially visible at the bottom.

Red Hat OpenShift

Plataforma central

- Inicio
- Favoritos
- Ecosistema
- Cargas de trabajo
- Redes
- Almacenamiento
- Compilaciones
- Canalizaciones
- Observar
- Calcular
- Gestión de usuarios
- Administración
- Service Mesh
- Integration Platform
 - Overview
 - Integration Flows
 - Platform Status

Platform Status

6/7 services healthy - Last checked: 22:39:04

Refresh

9	0	209	1	6/7
Integration Flows	PipelineRuns	ArgoCD ApplicationSets	Ephemeral (Quick Try) Create Quick Try flow	Services Healthy

External Services

Integration Operator Quarkus operator reconciling IntegrationFlow CRs	HEALTHY 1/1 replicas ready
Console Plugin OpenShift Console Dynamic Plugin serving this UI	HEALTHY 1/1 replicas
Kaoto Designer Visual editor for Apache Camel integrations	HEALTHY 1/1 replicas
Gitea (Git Server) Hosts scaffolded worker source code repositories	HEALTHY 1 replicas in gitea namespace
ArgoCD (GitOps) Deploys workers to target clusters via ApplicationSets	HEALTHY Server: 1 replicas
Tekton Pipeline CI pipeline for building worker container images	ERROR Pipeline not found
OpenTelemetry Collector Receives traces and metrics from worker pods	HEALTHY 1 replicas

Platform Pods (19)

Health: Operator · Kaoto · OTel · (GitOps services when enabled)

Demo · TTL & lifecycle

The screenshot displays the Red Hat OpenShift Platform Central interface. On the left is a sidebar with navigation links: Inicio, Favoritos, Ecosistema, Cargas de trabajo, Redes, Almacenamiento, Compilaciones, Canalizaciones, Observar, Calcular, Gestión de usuarios, Administración, Service Mesh, and Integration Platform. The main area shows the configuration for a flow named 'parallel-enrichment', which is currently in a 'Running' state. The configuration is shown in the 'Spec' tab, displaying a YAML snippet for a Camel route. On the right, there are several panels: 'Node Telemetry' showing metrics for different components (http-source-1, log-processor, kafka-sink, transform-1), 'Resources' with links to Logs, Runtime Pods, Build Pods, PipelineRuns, ArgoCD App, and Git Repository, 'Target Clusters' (None), 'State' (running), 'Last Commit' (585ae46), and 'Message' (ApplicationSet reconciled, tracking 1 cluster deployment(s)). At the top right of the main area, there are buttons for 'Pause', 'Stop', and 'Save Design'.

Red Hat OpenShift

Plataforma central

Inicio

Favoritos

Ecosistema

Cargas de trabajo

Redes

Almacenamiento

Compilaciones

Canalizaciones

Observar

Calcular

Gestión de usuarios

Administración

Service Mesh

Integration Platform

parallel-enrichment Running GitOps

Engine: CAMEL_ROUTE · Branch: main

Visual Flow Kaoto Designer YAML Editor History Dependencies Spec Logs Status

```
{
  "branch": "main",
  "engine": "CAMEL",
  "gitRepository": "https://gitea.example.com/user1/parallel-enrichment",
  "integrationType": "CAMEL_ROUTE",
  "kaotoDesign": "- route:\n id: parallel-enrichment\n from:\n uri: \"platform-http://api/enrich\"\n steps:\n - log:\n message: \"Enriching: ${body}\"
",
  "targeting": {
    "clusters": [
      "local"
    ],
    "strategy": "explicit"
  }
}
```

Node Telemetry

http-source-1	62ms	4768rpm	●
log-processor	19ms	2119rpm	●
kafka-sink	191ms	4601rpm	●
transform-1	114ms	346rpm	●

Updated: 22:46:24

Resources

- [Logs](#)
- [Runtime Pods](#)
- [Build Pods](#)
- [PipelineRuns](#)
- [ArgoCD App](#)
- [Git Repository](#)

Target Clusters

None

State

running

Last Commit

585ae46

Message

ApplicationSet reconciled, tracking 1 cluster deployment(s)

ttlSeconds · extend from console · optional promote-to-gitops (not live)

Try it yourself

1. **Joke 101** — [tutorial Getting Started](#) · OpenShift Local · internal registry ·

`operator-sdk`

2. **Next** — OperatorHub → OpenShift Integration Operator · ephemeral YAML

3. **Publish** — PR to [community-operators](#) when ready

Also fine: ROSA / ARO / on-prem · or **kind** + `operator-sdk` (no Console plugin)

What you learned

1. Operators extend K8s with CRDs + reconcile loops
2. QOSDK scaffolds from code.quarkus.io — Java, not Go
3. Joke 101 — bundle + operator-sdk on OpenShift Local (internal registry)
4. Community catalog — same path → PR to OperatorHub; anyone can contribute
5. Next level: Integration Operator · ephemeral Quick Try

Links & resources

Getting Started — [Podman · OpenShift Local · Joke 101](#)

OpenShift Local — developers.redhat.com/products/openshift-local

QOSDK docs — docs.quarkiverse.io/quarkus-operator-sdk

Joke sample (101) — [quarkiverse/.../samples/joke](https://quarkiverse.github.io/quarkiverse-docs/samples/joke)

Next step — [OpenShift Integration Operator](#)

Contribute — [community-operators](#) (OperatorHub catalog)

Quarkus Club — [YouTube live](#)

Thank you

Quarkus Club

BUILD OPERATORS IN JAVA · OPEN SOURCE

[LinkedIn](#) · [GitHub](#)

